Reviewer Pack — Minimal Rank of Hodge Structures Bounds Resolution Proof Len…

Entry 86cad8a62f93 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 07:52:34 UTC

Minimal Rank of Hodge Structures Bounds Resolution Proof Length for Tseitin Formulas

Entry ID: `86cad8a62f93` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 07:52:34 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For a given Tseitin formula on n variables, the length of its Resolution proof is at least 2^Ω(minRank(H(G))), where H(G) is the Hodge structure associated with the graph G representing the Tseitin formula.', 'Here, minRank(H(G)) denotes the minimum rank among all irreducible components of the Hodge structure H(G).', 'The conjecture holds for all n ≤ 40 and must be proven using only pure Python without external solvers.']

Rationale (proposer's reasoning):

> ['Hodge structures provide a rich algebraic framework to study geometric properties of complex varieties. By linking this with Tseitin formulas, which are graph-based boolean formulas, we may uncover new structural insights into the complexity of Resolution proofs.', 'Previous research has shown that certain graph invariants can bound the size of Resolution proofs for specific types of formulas. This conjecture explores whether Hodge structures offer a similar bounding power.', 'The connection between algebraic geometry and computational complexity is underexplored, suggesting potential for new results.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b7311290abfeba0a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n ≤ 40, the mean of minRank(H(G)) values correlates with a lower bound of 2^Ω(minRank(H(G))) on Resolution proof lengths across at least 10 independent random Tseitin formulas. The criterion is falsified if any single instance does not meet this correlation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge structure" AND "Resolution proof complexity" - "algebraic geometry" AND Tseitin formulas AND resolution proof length" - "minimal rank Hodge structures" AND bounds AND proof complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=212.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n + 1)]
        clauses = []
        for x in variables:
            clauses.append(f'{x} | ~{x}')
        for i in range(2 ** (n - 1)):
            clause = ' & '.join([f'~{variables[j]}' if (i >> j) & 1 else variables[j] for j in ran
            clauses.append(clause)
        return clauses

    def construct_graph(formula):
        graph = {}
        for clause in formula:
            literals = [l.strip('~') for l in clause.split(' | ') if l]
            for i in range(len(literals)):
                for j in range(i + 1, len(literals)):
                    lit_i = literals[i]
                    lit_j = literals[j]
                    if lit_i not in graph:
                        graph[lit_i] = set()
                    if lit_j not in graph:
                        graph[lit_j] = set()
                    graph[lit_i].add(lit_j)
                    graph[lit_j].add(lit_i)
```

```
        return graph

    def compute_hodge_structure(graph):
        # Placeholder for Hodge structure computation
        # This is a dummy implementation and should be replaced with actual logic
        hodge_structure = {}
        for node in graph:
            if node not in hodge_structure:
                hodge_structure[node] = 1
        return hodge_structure

    def min_rank(hodge_structure):
        return min(hodge_structure.values())

    def resolution_proof_length(formula):
        # Placeholder for Resolution proof length computation
        # This is a dummy implementation and should be replaced with actual logic
        return len(formula)

    n = random.randint(5, 40)
    formula = generate_tseitin_formula(n)
    graph = construct_graph(formula)
    hodge_structure = compute_hodge_structure(graph)
    min_rank_value = min_rank(hodge_structure)
    proof_length = resolution_proof_length(formula)

    return {
        "metric_name": "Resolution Proof Length",
        "metric_value": proof_length,
        "instances_tested": 1,
        "conjecture_holds": proof_length >= 2 ** (min_rank_value * math.log(2)),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [random.getrandbits(32) for _

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
(empty)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes successfully and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12175
2	preregistration	ollama_remote	glm4:latest	0	0	6397
3	novelty	ollama_remote	glm4:latest	0	0	4595
4	novelty	ollama_remote	glm4:latest	0	0	5441
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14141
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14778
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9244
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9618
9	judge	ollama_remote	glm4:latest	0	0	57922

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 134311 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/86cad8a62f93.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/86cad8a62f93.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/86cad8a62f93.tar.gz` (if generated)